Clase 3 (Unidad 2) — Modelado de datos y REST con Django REST Framework

Duración total: 4 horas | Proyecto: modelos Project/Task/Tag + primeros serializers

Parte 0 — Antes de empezar (5 min)

Abran `taskflow/` en VS Code, terminal nueva, activen el entorno virtual (`source venv/bin/activate` o `venv\Scripts\Activate.ps1`) y confirmen `(venv)` en el prompt. Si quedó corriendo un `runserver` de la clase anterior en otra terminal, pueden dejarlo o reiniciarlo con `Ctrl+C` y volverlo a levantar más adelante.

Parte 1 — Principios REST y por qué DRF (40 min)

1.1 ¿Qué es REST?

REST (*Representational State Transfer*) es un **estilo arquitectónico**, no una librería ni un paquete que se instale. Sus reglas prácticas más importantes:

- Cada **recurso** (una tarea, un proyecto) tiene una URL propia: `/api/tasks/5/` identifica **una** tarea puntual, la de `id=5`.
- El **método HTTP** indica la acción sobre ese recurso: GET (leer), POST (crear), PUT/PATCH (actualizar), DELETE (borrar). Profundizamos los códigos de estado asociados en la Clase 4.
- El servidor no guarda estado de conversación entre peticiones (**stateless**): cada petición trae, en sus headers o su body, toda la información necesaria para procesarla (por ejemplo, el token de autenticación, que vemos en la Unidad 3).

1.2 El problema que resuelve DRF

Si tuviéramos que hacerlo a mano con Django puro (como el `health_check` de la Clase 1), por cada modelo repetiríamos: convertir objetos del ORM a diccionarios, convertir esos diccionarios a texto JSON, validar el JSON entrante campo por campo, elegir el código de estado HTTP correcto para cada caso... **Django REST Framework (DRF)** automatiza todo esto sobre tres piezas que vamos a ir incorporando: **Serializers** (hoy), **Views/ViewSets** (Clase 4) y **Routers** (Clase 4).

Parte 2 — Modelado: Project, Task y Tag (85 min)

2.1 Diseño del modelo de datos

Vamos a reemplazar el modelo `Task` simple de la Clase 2 por un modelo relacional real:

- Un **Project** agrupa muchas **Task** (relación **uno a muchos**: un proyecto tiene muchas tareas, pero cada tarea pertenece a un único proyecto).
- Una **Task** puede tener muchos **Tag** (etiquetas), y un mismo `Tag` puede estar en muchas tareas distintas → relación **muchos a muchos** (many-to-many).

Abran `taskflow/core/models.py` y **reemplacen todo el contenido** por:

```
from django.db import models


class Project(models.Model):
    name = models.CharField(max_length=150)
    description = models.TextField(blank=True)
    created_at = models.DateTimeField(auto_now_add=True)

    def __str__(self):
        return self.name


class Tag(models.Model):
    name = models.CharField(max_length=50, unique=True)

    def __str__(self):
```

```
        return self.name


class Task(models.Model):
    PRIORITY_CHOICES = [("baja", "Baja"), ("media", "Media"), ("alta", "Alta")]
    STATUS_CHOICES = [
        ("pendiente", "Pendiente"),
        ("en_progreso", "En progreso"),
        ("completada", "Completada"),
    ]

    project = models.ForeignKey(Project, on_delete=models.CASCADE, related_name="tasks")
    title = models.CharField(max_length=200)
    description = models.TextField(blank=True)
    priority = models.CharField(max_length=10, choices=PRIORITY_CHOICES, default="media")
    status = models.CharField(max_length=15, choices=STATUS_CHOICES, default="pendiente")
    due_date = models.DateField(null=True, blank=True)
    tags = models.ManyToManyField(Tag, blank=True, related_name="tasks")
    created_at = models.DateTimeField(auto_now_add=True)

    def __str__(self):
        return f"{self.title} ({self.project.name})"
```

Detalle de los campos nuevos: `TextField` es como `CharField` pero sin límite de longitud, pensado para textos largos (`description`). `ForeignKey(Project, on_delete=models.CASCADE, related_name="tasks")` es la relación uno-a-muchos: `on_delete=models.CASCADE` significa "si se borra el Project, se borran automáticamente todas sus Task asociadas"; `related_name="tasks"` es el nombre que vamos a usar para ir "hacia atrás" desde un proyecto (`mi_project.tasks.all()`), en vez del nombre por defecto, menos claro, que generaría Django. `DateField(null=True, blank=True)` permite que la fecha límite quede vacía: `null=True` a nivel base de datos, `blank=True` a nivel de validación de formularios/API. `ManyToManyField(Tag, ...)` crea automáticamente, "por detrás", una tabla intermedia que vincula tareas con etiquetas.

2.2 Migraciones (y qué hacer si hay conflicto con datos previos)

Como cambiamos la forma del modelo `Task` respecto a la Clase 2, lo más simple en esta etapa de desarrollo (todavía no importa perder los datos de prueba) es reiniciar la base y el historial de migraciones. Parados en `taskflow/`:

```
rm db.sqlite3
rm core/migrations/0001_initial.py
```

(En Windows PowerShell: `Remove-Item db.sqlite3` y `Remove-Item core\migrations\0001_initial.py`). Después:

```
python manage.py makemigrations core
python manage.py migrate
```

Esto genera una migración `0001_initial.py` completamente nueva, que ya incluye `Project`, `Tag` y el `Task` relacional, y la aplica sobre una base de datos SQLite recién creada.

2.3 Registrar en el admin

Abran `taskflow/core/admin.py` y reemplacen el contenido por:

```
from django.contrib import admin
from .models import Project, Task, Tag

admin.site.register(Project)
admin.site.register(Task)
admin.site.register(Tag)
```

Si no tienen un superusuario todavía (se perdió al borrar `db.sqlite3`), vuelvan a crearlo:

```
python manage.py createsuperuser
```

Levanten el servidor (`python manage.py runserver`) y, desde `http://127.0.0.1:8000/admin/`, carguen a mano 2 proyectos con 3-4 tareas cada uno, y algunos tags (por ejemplo: "backend", "urgente", "documentación").

Parte 3 — Serializers de solo lectura (75 min)

3.1 Confirmar la instalación de DRF

Ya instalamos `djangorestframework` en la Clase 1 y lo agregamos a `INSTALLED_APPS`. Verifiquen en `taskflow/config/settings.py` que la lista incluya `"rest_framework"` y `"core"`.

3.2 El primer serializer

Un **serializer** de DRF cumple dos roles: transforma instancias de modelo → JSON (**serialización**, lo que usamos hoy) y valida JSON entrante → datos de modelo (**deserialización**, la usamos desde la Clase 4 para crear/editar).

Creen el archivo `taskflow/core/serializers.py` (no existe todavía):

```
from rest_framework import serializers
from .models import Project, Task, Tag


class TagSerializer(serializers.ModelSerializer):
    class Meta:
        model = Tag
        fields = ["id", "name"]


class TaskSerializer(serializers.ModelSerializer):
    tags = TagSerializer(many=True, read_only=True)

    class Meta:
        model = Task
        fields = [
            "id", "project", "title", "description",
            "priority", "status", "due_date", "tags", "created_at",
        ]


class ProjectSerializer(serializers.ModelSerializer):
    tasks = TaskSerializer(many=True, read_only=True)

    class Meta:
        model = Project
        fields = ["id", "name", "description", "tasks", "created_at"]
```

`ModelSerializer` es una clase de DRF que, a partir de la clase `Meta` (donde indicamos `model` y `fields`), genera automáticamente los campos del serializer inspeccionando el modelo — evita repetir a mano cada campo con su tipo. `fields` es la lista explícita de qué columnas exponer en el JSON (evitamos `fields = "__all__"` a propósito, para tener control fino de qué se expone). `tags = TagSerializer(many=True, read_only=True)` **sobreescribe** el campo automático de `tags` para que, en vez de devolver solo los IDs de las etiquetas, devuelva el objeto completo serializado (`{"id":1,"name":"urgente"}`) — `many=True` porque es una relación con múltiples elementos, `read_only=True` porque hoy solo leemos, no permitimos crear tags desde acá.

> 💡 **Nota del docente**
> Fíjense que `ProjectSerializer` "anida" a `TaskSerializer`, que a su vez anida a `TagSerializer`. DRF arma automáticamente el JSON anidado recorriendo esas relaciones — el resultado final para un proyecto va a incluir sus tareas, y cada tarea sus etiquetas, todo en una sola respuesta.

3.3 Vistas de solo lectura basadas en funciones

Para entender qué hace DRF paso a paso (antes de usar `ViewSet`, que automatiza aún más, en la Clase 4), escribimos vistas simples con el decorador `@api_view`. Reemplacen el contenido de `taskflow/core/views.py` (conservando `health_check`, que ya tenían):

```
from django.http import JsonResponse
from rest_framework.decorators import api_view
from rest_framework.response import Response
from .models import Project, Task
from .serializers import ProjectSerializer, TaskSerializer


def health_check(request):
    return JsonResponse({"status": "ok", "service": "TaskFlow API"})


@api_view(["GET"])
def project_list(request):
    projects = Project.objects.all()
    serializer = ProjectSerializer(projects, many=True)
    return Response(serializer.data)


@api_view(["GET"])
def task_list(request):
    tasks = Task.objects.select_related("project").prefetch_related("tags").all()
    serializer = TaskSerializer(tasks, many=True)
    return Response(serializer.data)
```

`@api_view(["GET"])` es un **decorador**: convierte una función normal de Python en una vista de DRF, restringiendo qué métodos HTTP acepta (acá, solo GET; cualquier otro método devolvería automáticamente un `405 Method Not Allowed`). `Project.objects.all()` es una **consulta del ORM**: trae todas las filas de la tabla `Project`. `ProjectSerializer(projects, many=True)` instancia el serializer, pasándole el queryset y `many=True` porque son varios objetos (no uno solo); `serializer.data` ejecuta la conversión y devuelve la estructura lista para convertirse en JSON. `Response(...)` es la clase de DRF (similar a `JsonResponse`, pero integrada con el resto del framework) que arma la respuesta HTTP final.

Actualicen `taskflow/core/urls.py`:

```
from django.urls import path
from . import views

urlpatterns = [
    path("health/", views.health_check, name="health_check"),
    path("projects/", views.project_list, name="project_list"),
    path("tasks/", views.task_list, name="task_list"),
]
```

Reinicien el servidor si hace falta y visiten `http://127.0.0.1:8000/api/projects/` y `http://127.0.0.1:8000/api/tasks/`: DRF muestra una interfaz navegable (la ***Browsable API***) con el JSON formateado y coloreado, muy útil para probar mientras desarrollamos, sin necesidad de herramientas externas.

💡 **Nota del docente**
`select_related("project")` y `prefetch_related("tags")` evitan el problema de **N+1 queries**: sin ellos, Django ejecutaría una consulta SQL adicional a la base de datos por cada tarea, para traer su proyecto (`select_related`, usado en relaciones uno-a-uno o muchos-a-uno) y sus tags (`prefetch_related`, usado en relaciones muchos-a-muchos o uno-a-muchos inversas). Con 100 tareas, la diferencia es 2 consultas contra 100+.

Parte 4 — Guardar el avance en Git y GitHub (20 min)

Parados en `taskflow/`:

```
git status
```

Deberían ver modificados `core/models.py`, `core/admin.py`, `core/views.py`, `core/urls.py`, y como nuevo `core/serializers.py`, además de una nueva migración en `core/migrations/`.

```
git add .
git commit -m "Modela Project/Task/Tag con relaciones y expone endpoints de solo lectura con DRF"
git push
```

Parte 5 — Cierre y tarea

5.1 Resumen

- Modelado relacional: `ForeignKey` (uno a muchos) y `ManyToManyField` (muchos a muchos).
- Serializers anidados de DRF (`ModelSerializer`) para solo lectura.
- Primeras vistas con `@api_view` y la Browsable API.

5.2 Tarea para la casa

1. Agreguen un endpoint `GET /api/projects/<id>/` que devuelva un único proyecto (busquen en la documentación de Django la función `get_object_or_404` y cómo recibir un parámetro de la URL con `<int:project_id>`).
2. Investiguen qué son `select_related` y `prefetch_related` y en qué se diferencian internamente (tipo de SQL que generan).
3. Guarden el avance repitiendo el flujo de la Parte 4.

5.3 Próxima clase

Clase 4: convertimos estas vistas de solo lectura en un **CRUD completo** con `ModelViewSet`, códigos de estado HTTP y validaciones de negocio, y un manejo centralizado de errores.

Diccionario de términos de esta clase

- **Browsable API:** interfaz web navegable que DRF genera automáticamente para probar los endpoints desde el navegador.
- **CRUD:** las cuatro operaciones básicas sobre datos — Create (crear), Read (leer), Update (actualizar), Delete (borrar).
- **DRF (Django REST Framework):** paquete adicional sobre Django especializado en construir APIs.
- **ForeignKey:** campo de un modelo que crea una relación "muchos a uno" hacia otro modelo (muchas tareas pueden pertenecer a un mismo proyecto).
- **ManyToManyField:** campo que crea una relación "muchos a muchos" entre dos modelos (una tarea puede tener varias etiquetas, y una etiqueta estar en varias tareas).
- **N+1 queries:** problema de rendimiento en el que, por no optimizar, se ejecuta una consulta SQL extra por cada elemento de una lista.
- **REST (Representational State Transfer):** estilo arquitectónico para diseñar APIs, basado en recursos identificados por URL y verbos HTTP.
- **Serializer:** clase de DRF que convierte instancias de modelo a JSON (y valida JSON entrante para convertirlo en datos de modelo).
- **Stateless (sin estado):** característica de REST por la cual el servidor no recuerda nada de peticiones anteriores; cada petición es independiente.